GRADE 6 · CODING CLUB · LESSON 3

FastAPI

Turn a few lines of Python into a real web API

And get a free, clickable instruction page!

TODAY

1. What is FastAPI?
2. Hello, API!
3. Run it
4. Path parameters
5. Query parameters
6. The automatic /docs page
7. Build: Fun Facts API
8. Quiz

What is FastAPI?

- A free Python library for building **web APIs**
- You write normal Python **functions** — FastAPI turns them into **endpoints**
- It uses **Pydantic** to check the data (our Checker from Lesson 2!)
- It writes the **menu** for you: an automatic, interactive **docs page**
- It's fast, and it's used by real companies
- Remember the restaurant: FastAPI is the **waiter**, your functions are the **kitchen**

TALKING POINTS

Recap the waiter analogy. Emphasise: we don't have to build the waiter — FastAPI is the waiter. We just cook (write functions).

Hello, API!

Save this as `main.py`

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello from my first API!"}
```

`app = FastAPI()`
makes the API

`@app.get("/")`
"when someone sends a GET request to /, run the function below"

`return {...}`
the dictionary becomes **JSON**

TALKING POINTS
The @ line is called a decorator — like a sticker that says 'this function is the answer for this address'. Kids don't need the word, just the idea.

Run it!

- Open a terminal in your `my_api` folder and type:
  - `fastapi dev main.py`
- Open your browser at `http://127.0.0.1:8000`
  - You'll see: `{"message": "Hello from my first API!"}`
- **dev** mode reloads automatically every time you **save** your file
- Stop the API with **Ctrl + C**
- 127.0.0.1 is **your own computer** — your API is running right there!

TALKING POINTS

Celebrate the first working response — it's a big moment. Common problems: wrong folder, file not saved, typo in 'main.py'. Change the message, save, and refresh the browser to show auto-reload.

Path parameters: values inside the address

Curly brackets { } in the path become function inputs

```
@app.get("/hello/{name}")
def hello(name: str):
    return {"greeting": f"Hello {name}!"}

@app.get("/square/{number}")
def square(number: int):
    return {"number": number, "square": number * number}
```

Try in the browser:

`/hello/Mia`

`/square/7`

`/square/seven` ← what happens?

TALKING POINTS

/square/seven gives a 422 error saying it should be a valid integer — because number: int is a type hint, and FastAPI uses Pydantic to check it. No extra code needed! Link back to Lesson 2.

Query parameters: extra options after the ?

Function inputs that are **not** in the path become **query parameters**. A default value makes them optional.

```
@app.get("/hello/{name}")
def hello(name: str, times: int = 1):
    return {"greeting": f"Hello {name}! " * times}

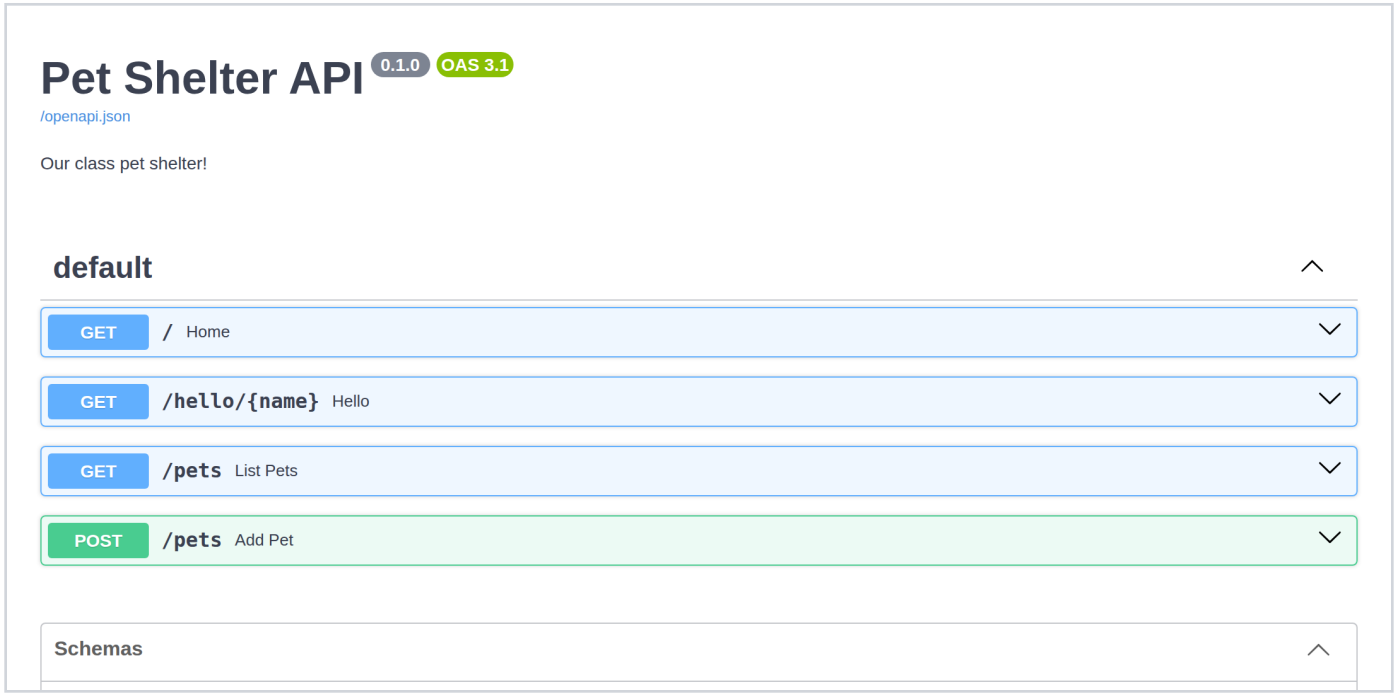
# /hello/Mia           -> {"greeting": "Hello Mia! "}
# /hello/Mia?times=3   -> {"greeting": "Hello Mia! Hello Mia! Hello Mia! "}
# /hello/Mia?times=abc -> error 422: "Input should be a valid integer"
```


TALKING POINTS

These outputs are real. Recall the URL slide from Lesson 1: path vs query. Ask: what would a ?shout=true option do? Let fast finishers add it.

The automatic docs page: /docs

Go to `http://127.0.0.1:8000/docs` — FastAPI wrote this **menu** for you!



Every endpoint is listed

GET = blue
POST = green

Click one, press **Try it out**, fill in values, press **Execute**

There's a second style at `/redoc`

TALKING POINTS

This screenshot is from the Pet Shelter API we build in Lesson 4. Open the docs for the kids' own app live. Explain why docs matter: other programmers (or your future self) need to know what the API can do — and FastAPI keeps the docs up to date automatically whenever you change your code.

Build: Fun Facts API (25 minutes)

- Make a list of 5 fun facts about anything you like
- Add these endpoints:
  - `GET /` → a welcome message
  - `GET /facts` → all your facts
  - `GET /facts/{number}` → one fact (0 is the first!)
  - `GET /random` → a random fact (`import random`, `random.choice(facts)`)
- Test every endpoint in **/docs** with Try it out
- Stretch: `?shout=true` makes the fact UPPERCASE

TALKING POINTS

What happens with /facts/99? The program crashes with an error 500 — a great discussion: our API should say 'not found' politely. We'll fix that in Lesson 4 with HTTPException. Stretch: shout: bool = False, then fact.upper() if shout.

Quiz time!

- 1. What command starts your API?
- 2. In `@app.get("/square/{number}")`, what is `{number}`?
- 3. Why does `/square/seven` give an error, even though we didn't write any checking code?
- 4. What can you do on the `/docs` page?
- **Next lesson:** the **Pet Shelter API** — adding new data with POST!

TALKING POINTS

Answers: 1) fastapi dev main.py. 2) A path parameter — a value taken from the address. 3) number: int tells FastAPI it must be a whole number, and FastAPI uses Pydantic to check it. 4) See every endpoint and test it with Try it out.